

1. Bubble Sort - Sort Student Marks

Problem Statement:

A school maintains student marks in an array. Implement **Bubble Sort** to sort the student marks in **ascending order**.

Hint:

- Traverse through the array multiple times.
 - Compare adjacent elements and swap if needed.
 - Repeat the process until no swaps are required.
-

2. Insertion Sort - Sort Employee IDs

Problem Statement:

A company stores **employee IDs** in an unsorted array. Implement **Insertion Sort** to sort the employee IDs in **ascending order**.

Hint:

- Divide the array into sorted and unsorted parts.
 - Pick an element from the unsorted part and insert it into its correct position in the sorted part.
 - Repeat for all elements.
-

3. Merge Sort - Sort an Array of Book Prices

Problem Statement:

A bookstore maintains a list of book prices in an array. Implement **Merge Sort** to sort the prices in **ascending order**.

Hint:

- **Divide** the array into two halves recursively.
- **Sort** both halves individually.
- **Merge** the sorted halves by comparing elements.

4. Quick Sort - Sort Product Prices

Problem Statement:

An e-commerce company wants to display product prices in **ascending order**. Implement **Quick Sort** to sort the product prices.

Hint:

- Pick a **pivot** element (first, last, or random).
 - **Partition** the array such that elements smaller than the pivot are on the left and larger ones are on the right.
 - Recursively apply Quick Sort on left and right partitions.
-

5. Selection Sort - Sort Exam Scores

Problem Statement:

A university needs to sort students' **exam scores** in ascending order. Implement **Selection Sort** to achieve this.

Hint:

- Find the **minimum element** in the array.
 - Swap it with the first unsorted element.
 - Repeat the process for the remaining elements.
-

6. Heap Sort - Sort Job Applicants by Salary

Problem Statement:

A company receives job applications with different **expected salary demands**. Implement **Heap Sort** to sort these salary demands in **ascending order**.

Hint:

- Build a **Max Heap** from the array.
- Extract the largest element (root) and place it at the end.
- Reheapify the remaining elements and repeat until sorted.

7. Counting Sort - Sort Student Ages

Problem Statement:

A school collects students' **ages** (ranging from 10 to 18) and wants them sorted. Implement **Counting Sort** for this task.

Hint:

- Create a **count array** to store the frequency of each age.
- Compute cumulative frequencies to determine positions.
- Place elements in their correct positions in the output array.